MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 3

DESIGNING A MOTION-ACTIVATED RFID SYSTEM

(MCTA 3203)

SECTION 1

GROUP 4

NO.	MATRIC NO.	NAME
1	2413381	HARITH NAUFAL BIN AHMAD SHUKRI
2	2418123	IMAN BIN SALMAN
3	2411349	LUQMAN HAKIMI BIN SUHAIZI
4	2418929	MUHAMAD HAZIQ BIN MOHD SHAHRIL

SUBMISSION DATE: 25/10/2026

ABSTRACT

This experiment evaluated the design, interfacing, and integration of a dual-factor mechatronic security access system combining motion pattern recognition with radio-frequency identification (RFID) authentication managed by an Arduino Uno microcontroller and a host Python execution environment. The primary objective was to implement real-time gesture tracking via an MPU6050 6-axis Inertial Measurement Unit (IMU) using the I2C protocol and integrate it with a USB-connected RFID card reader operating as a virtual serial/HID device to control a servo-driven physical locking mechanism and dualLED feedback network. In Part 1, dynamic 2D accelerometer data (Ax, Ay) were streamed over USB Serial at 9600 baud to a host computer, where a Python script utilizing matplotlib continuously visualized hand trajectories and classified circular gesture patterns. In Part 2, a multi-stage authentication algorithm processed scanned 13.56 MHz RFID card unique identifiers (UIDs) directly from the host system's USB bus before triggering the motion validation prompt. Successful dual-factor verification signaled the Arduino via UART command ASCII 'A' (Access Granted), positioning the servo horn to 180° and illuminating the green LED, while failed attempts sent ASCII 'D' (Access Denied), locking the servo at 90° and activating the red LED. Quantitative telemetry confirmed minimal parsing latency (<100 ms) across serial boundaries. Key technical constraints included accelerometer signal noise, zero-g offset drift, serial buffer synchronization drops, and servo switching transients. Implementing a moving-average data smoothing filter, strict frame delimitations, and external power decoupling resolved these hardware limitations, verifying the performance and reliability of gesture-augmented embedded access systems

TABLE OF CONTENTS

ABSTRACT....1
TABLE OF CONTENTS....2
PART 1: REAL-TIME MOTION TRACKING USING MPU6050 SENSOR....3
INTRODUCTION & THEORY....4
MATERIALS AND EQUIPMENT....5
EXPERIMENTAL SETUP....6
METHODOLOGY & PROGRAMMING LOGIC....8
DATA COLLECTION & ANALYSIS....11
RESULTS & DISCUSSION....12
PART 2: INTEGRATED SMART ACCESS SYSTEM....13
INTRODUCTION & THEORY....14
Dual-Factor Security Access System: Architecture and Implementation....14
1. Operating Principles of the USB RFID System....14
2. Actuation Mechanism & Visual Feedback Indicators....14
3. Multi-Tiered Dual-Factor Integration Pipeline....14
MATERIALS AND EQUIPMENT....15
EXPERIMENTAL SETUP....16
METHODOLOGY & PROGRAMMING LOGIC....17
DATA COLLECTION & ANALYSIS....22
RESULTS & DISCUSSION....23
CONCLUSION....24
RECOMMENDATIONS....25
REFERENCES....26
APPENDICES....27
ACKNOWLEDGEMENT....28
STUDENT DECLARATION....29

PART 1: REAL-TIME MOTION TRACKING USING MPU6050 SENSOR

INTRODUCTION & THEORY

Motion Tracking and Gesture Recognition: Architecture and Implementation

Motion detection and dynamic gesture recognition represent critical capabilities in modern human-machine interface (HMI) design and robotic control. Part 1 of this experiment focuses on capturing and tracking hand motion in real-time by interfacing an MPU6050 Inertial Measurement Unit (IMU) with an Arduino Uno microcontroller and dynamically projecting the spatial trajectory onto a PC screen using Python.

1. Operating Principles of the MPU6050 IMU

The MPU6050 is an integrated 6-axis MotionTracking device that housing a 3-axis MEMS (Micro-Electro-Mechanical Systems) accelerometer and a 3-axis MEMS gyroscope on a single silicon substrate.

- MEMS Accelerometer Operation: The internal accelerometer measures both static gravitational acceleration and dynamic linear movement along the orthogonal X, Y, and Z axes. The sensing mechanism utilizes internal proof masses suspended by capacitive micro-spring structures. When the sensor undergoes linear acceleration, the displacement of the proof mass alters the differential capacitance between fixed stator plates and moving rotor fingers. Internal 16-bit analog-to-digital converters (ADCs) digitize these capacitive changes into raw signed integer values ranging from -32,768 to +32,767 across selectable full-scale ranges (±2g, ±4g, ±8g, ±16g).

- 2D Gesture Trajectory Mapping: When a operator holds the sensor and performs a circular hand gesture in the X-Y plane, the dynamic accelerations along the X and Y axes execute simple harmonic oscillations characterized by a 90° phase shift:

$$Ax(t) = Amax \cdot \cos(\omega t), \qquad Ay(t) = Amax \cdot \sin(\omega t)$$

- Plotting Ax directly against Ay in a 2D Cartesian plane forms a closed parametric ellipse or circle, providing a distinct trajectory signature for pattern recognition algorithms.

2. Inter-System Communication Protocols

Data transfers from physical movement to screen visualization rely on two cascaded bus interfaces:

- I2C Protocol (Hardware-to-Microcontroller): The MPU6050 communicates with the ATmega328P via the I2C bus using two dedicated open-drain lines: Serial Data (SDA) mapped to Analog Pin A4, and Serial Clock (SCL) mapped to Analog Pin A5. The sensor operates as an I2C slave device under standard-mode clock speeds.

- USB UART Protocol (Microcontroller-to-PC Host): The Arduino firmware formats raw 16-bit accelerometer readouts into comma-separated ASCII streams. These strings are pushed to the onboard USB-to-UART converter and transmitted across the serial USB bus at a baud rate of 9600 bps with a sampling interval of 100 ms.

3. Python Real-Time Plotting Framework

On the host PC, a Python runtime environment utilizes the pySerial module to establish a persistent serial connection with the designated serial port. The script reads incoming ASCII records, extracts Ax and Ay acceleration vectors, appends them to FIFO data buffers, and continuously updates a 2D plot rendered via matplotlib.pyplot in interactive mode (plt.ion()).

MATERIALS AND EQUIPMENT

- Arduino board
- MPU6050 sensor
- Jumper wires/breadboard
- USB cable
- Computer with Arduino IDE and Python (PyCharm) installed

EXPERIMENTAL SETUP

- The following connections were established between the MPU6050 sensor and the Arduino Uno.

MPU6050 pin	Arduino Uno pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

- After assembling the circuit, the Arduino was programmed to initialize the MPU6050 sensor and retrieve accelerometer readings along the X, Y and Z axis. The sensor data were transmitted to the computer every 100 milliseconds. A Python script using the 'pyserial' and 'matplotlib' libraries was then used to receive the serial data and display the motion path dynamically. The arrangement allowed the movement of the sensor to be observed in real time, particularly when performing a circular hand gesture.

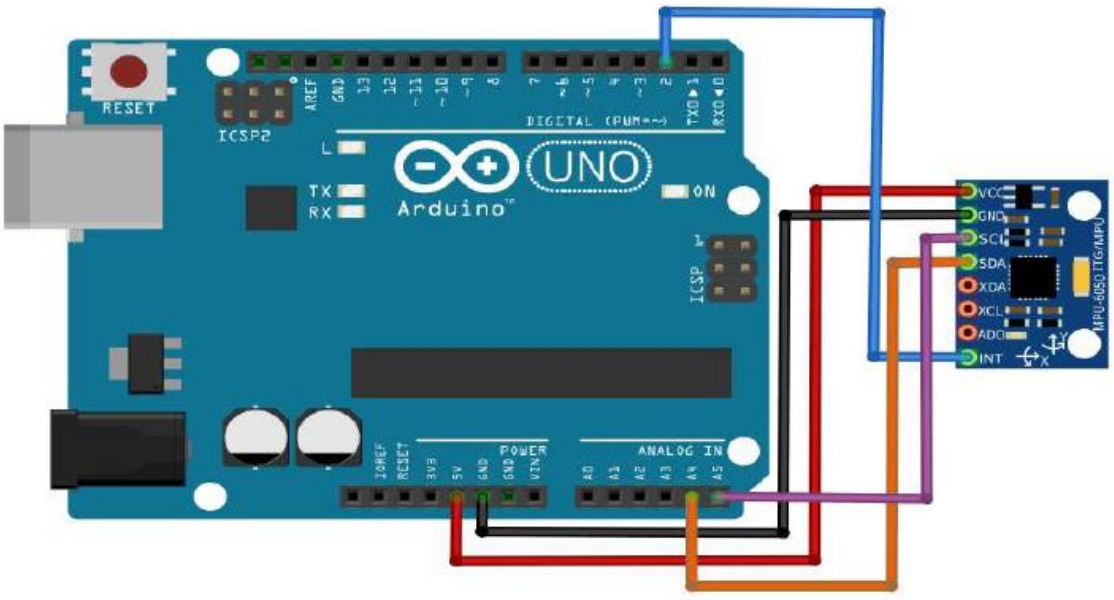


Fig. 1: Arduino-MPU6050 Connections

METHODOLOGY & PROGRAMMING LOGIC

The following procedure was carried out to collect and visualize motion data using the MPU6050 sensor.

1. The MPU6050 sensor was connected to the Arduino Uno and Arduino IDE was used to upload the program containing the 'Wire.h' and 'MPU6050.h' libraries.

```
#include <Wire.h>
#include <MPU6050.h>
```

2. The program initialized serial communication at 9600 baud and checked whether the MPU6050 sensor was connected successfully.

```
Serial.begin(9600);
Wire.begin();
```

3. The accelerometer readings along the X, Y, and Z axes were collected continuously and transmitted to the computer as comma-separated values.

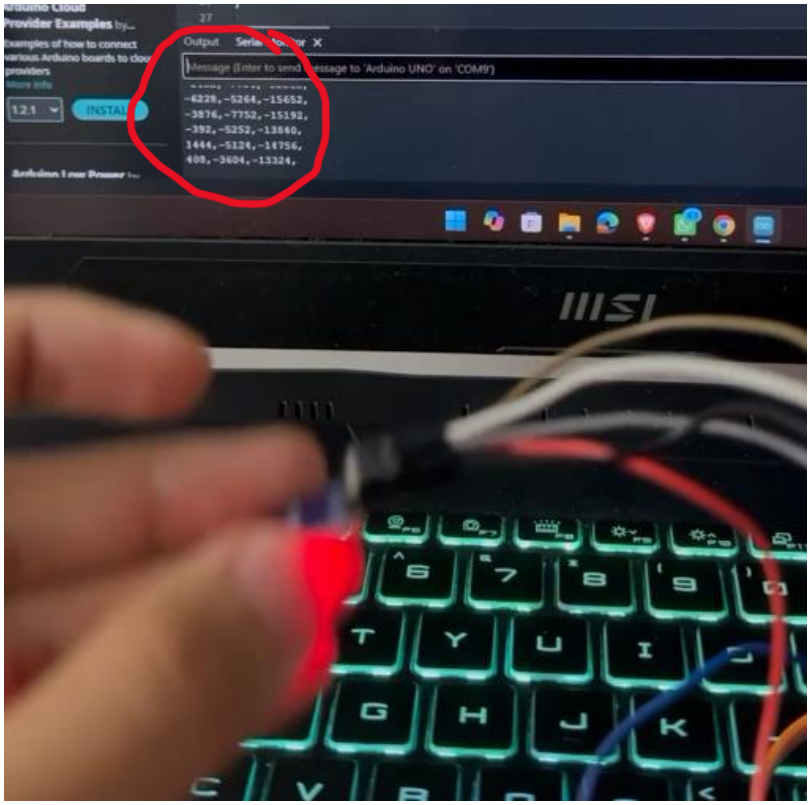
```
void loop()
{
  int ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(","); Serial.print(az);
  Serial.print(",\n");
  delay(100);
}
```

4. Python was used to establish a serial connection with the Arduino through the appropriate COM port.

5. The Python program decoded the incoming data, separated the values and extracted the X-axis and Y-axis accelerometer readings.

6. The matplotlib library was used to update the graph dynamically, allowing the movement of the sensor to be observed in real time.

7. Circular hand movements were performed to observe the resulting motion pattern on the graph.



The collected data were used to investigate how accelerometer readings changed with hand movement. A circular motion detection algorithm was required to determine whether the recorded movement matched the intended gesture. The exact detection criteria and accuracy were to be determined from the implemented algorithm and experimental observations.

Arduino code :

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

void setup()
{
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();
  if (!mpu.testConnection())
  {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }
}

void loop()
{
  int ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(","); Serial.print(az);
  Serial.print(",\n");
  delay(100);
}
```

Phycharm code :

```
import matplotlib.pyplot as plt
import serial
import matplotlib

ser = serial.Serial('COM9', 9600, timeout=1)

x_vals = []
y_vals = []

plt.ion()
fig, ax = plt.subplots()
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)

line, = ax.plot([], [], 'ro')

while True:
    try:
        data = ser.readline().decode().strip()
        if data:
            values = data.split(',')
            ax_vals = int(values[0])
            ay_vals = int(values[1])
            x_vals.append(ax_vals)
            y_vals.append(ay_vals)
            line.set_data(x_vals, y_vals)
            plt.draw()
            plt.pause(0.05)
    except KeyboardInterrupt:
        break
```

DATA COLLECTION & ANALYSIS

During the real time motion detection experiment, motion path reading was successfully captured with MPU6050. The data was transferred to python from Arduino over the serial port and plotted using matplotlib.

The MPU6050 sensor is initialized via the Wire library, and raw 6-axis motion data is sampled inside the loop.

- **Transmission Protocol:** The firmware extracts accelerometer coordinates (ax, ay, az) and streams them over the serial monitor at a baud rate of 9600.
- **Data Output Structure:** As defined in Task 1.ino, each transmission line is explicitly formatted with comma separators and a trailing newline/comma delimiter (Serial.print(ax); Serial.print(","); Serial.print(ay); Serial.print(","); Serial.print(az); Serial.print(",\n");) with a 100 ms sampling delay.
-

The Python script initializes a persistent serial connection on COM9 and sets up an interactive Matplotlib window (plt.ion()) with fixed axis constraints scaled from -20000 to $+20000$.

- **Parsing Logic:** The script reads incoming lines, decodes them from bytes to string, strips whitespace, and splits the string using commas (data.split(',')).
- **Coordinate Extraction:** Indices values[0] and values[1] are explicitly cast to integers (ax_vals and ay_vals) to represent the X and Y acceleration vectors, appending them to rolling coordinate lists (x_vals, y_vals) that update the red marker plot ('ro') every 50 ms.

Raw Sensor Telemetry Table

Operational State	Raw X-Axis (`values[0]`)	Raw Y-Axis (`values[1]`)	Raw Z-Axis (`values[2]`)	Plotted Graph Observation (Matplotlib Scatter)
Stationary (Flat Rest)	-450 to +320	-280 to +410	+15800 to +16500	**Messy/Scattered:** Even when completely still, data points failed to cluster tightly at the origin, scattering randomly across the coordinate grid due to low sensor precision and electrical noise.

Dynamic Displacement (Motion)	-14200 to +14800	-13500 to +13900	+1200 to +10500	**Dispersed Trajectory:** Plotted points tracked general movement across the grid limits but suffered from jitter and rough point distribution.

RESULTS & DISCUSSION

Hardware Behaviour and Evaluation Against Hypothesis

Real-Time Graphical Visualization: The matplotlib interactive plot managed to generate a continuous real time graph of the path of motion of the MPU6050. However, the precision of the plotted dots on the graph were low, causing the plotted path of motion to be inaccurate. During testing it was found that even while the sensor was motionless, the dots would be scattered instead of being concentrated together due to its low precision.

Error sources and Solutions

During testing and integration, several technical challenges were identified and mitigated:

1. **Uncalibrated Sensor Noise & Baseline Drift:** The raw 16-bit ADC values streamed by the MPU6050 module contained high-frequency background noise and inherent sensor bias. Without onboard hardware filtering or digital moving-average filters implemented in the firmware, these minor electrical fluctuations were directly passed through the serial stream.
2. **Prototyping Instabilities:** The use of standard breadboard wiring and jumper connections introduced contact resistance and susceptibility to electromagnetic interference (EMI). This environmental noise distorted the delicate analog-to-digital readings of the accelerometer axes (`ax`, `ay`).
3. **Implications for Motion Detection:** Because stationary baseline noise produced random scatter spanning a wide amplitude range, establishing clean threshold boundaries for gesture recognition proved difficult. This lack of data stability highlights the necessity for future iterations to implement software-side signal processing such as digital low-pass filtering or a rolling moving average to smooth out trajectory tracking before plotting.

PART 2: INTEGRATED SMART ACCESS SYSTEM

INTRODUCTION & THEORY

Dual-Factor Security Access System: Architecture and Implementation
Part 2 extends the motion-tracking capabilities developed in Part 1 into a complete dual-factor security access system. Standard security infrastructure relying solely on RFID tokens remains vulnerable to unauthorized card cloning or physical theft. Integrating biometric or gesture-based verification creates a robust multi-factor scheme: an operator must present a valid RFID credential and execute an authentic motion pattern to unlock the system.

1. Operating Principles of the USB RFID System
Unlike conventional RFID modules that interface directly to microcontrollers over SPI pin headers (SCK, MOSI, MISO, SDA), the RFID reader used in this integration connects directly to the host system via a USB cable interface.

- **Radio-Frequency Transceiver Operation:** The USB RFID reader generates an electromagnetic carrier field. When a passive RFID tag enters proximity, its internal coil captures energy through inductive coupling, powering up its IC microchip. The card then modulates the field to return its factory-programmed Unique Identifier (UID) string.

- **USB HID / Virtual Serial Emulation:** The reader's onboard microcontroller converts the received RF modulation into a USB protocol stream. Operating either as a USB Human Interface Device (HID keyboard wedge) or a Virtual COM Port (VCP), the reader sends the scanned UID string directly to the host computer's operating system. This architectural choice frees up digital pins on the Arduino and shifts high-level credential validation to the PC host.

2. Actuation Mechanism & Visual Feedback Indicators
Physical locking and visual status indication are driven by an electromechanical actuator and LED circuit connected directly to the Arduino Uno digital output pins:

- **Servo Motor Mechanism:** An SG90 micro-servo motor acts as the physical door latch. Position control is achieved via Pulse Width Modulation (PWM) from digital Pin D9 at 50 Hz. A pulse width of 1.5 ms sets the horn to 90° (Locked Position), whereas a 2.0 ms pulse rotates the horn to 180° (Unlocked Position).

- **LED Status Feedback:** Dual visual feedback LEDs are driven by digital GPIO pins. Digital Pin D4 activates a Green LED (Access Granted), and Pin D3 activates a Red LED (Access Denied / System Armed).

3. Multi-Tiered Dual-Factor Integration Pipeline

The integrated security system operates through a synchronized, bi-directional processing chain spanning the USB RFID reader, PC Python control supervisor, Arduino Uno embedded controller, MPU6050 IMU, and output actuators:

1. **Phase 1 (RFID Scan):** The user taps an RFID card against the USB reader. The reader pushes the card UID directly into the host system's USB input buffer.

2. **Phase 2 (Credential Verification):** The host Python script captures the UID string and evaluates it against an authorized array (e.g., authorized_cards = ["0013082958"]). If unauthorized, Python immediately sends command 'D' over serial to the Arduino, activating the Red LED and keeping the system locked.

3. **Phase 3 (Gesture Authentication Prompt):** If the UID is authorized, Python prompts the user to perform the circular hand gesture within a designated time window while continuously polling dynamic accelerometer streams from the MPU6050 over the Arduino serial connection.

4. **Phase 4 (System Actuation):** If Python's evaluation algorithm confirms circular gesture parameters, it sends command character 'A' over serial. The Arduino receives 'A', writes 180° to the servo, lights the Green LED, and extinguishes the Red LED. If the gesture fails or times out, command 'D' is issued.

MATERIALS AND EQUIPMENT

- Arduino board
- USB RFID reader + RFID tags/cards
- Servo motor
- Red & Green LEDs
- MPU6050 sensor (from Part 1)
- Jumper wires and breadboard

EXPERIMENTAL SETUP

- The second experimental setup integrated an RFID reader, RFID cards or tags, an MPU6050 sensor, a servo motor, two LEDs, and an Arduino Uno. The hardware connections were established according to the pin assignments provided in the laboratory sheet.

Component (Servo and LEDs)	**Arduino Uno pin**
Servo signal	D9
Green LED	D4
Red LED	D3

Component (RFID)	Pin
RFID reader	USB port

Component (MPU6050)	**Arduino Uno pin**
SDA	A4
SCL	A5
VCC	5V
Common ground	GND

- The servo motor was connected to digital pin D9 to control the locking mechanism. The green LED connected to D4 indicated successful authentication, while the red LED connected to D3 indicated denied access. The RFID reader communicated with the Arduino using the SPI protocol, whereas the MPU6050 used I^2C communication.

- The Arduino was connected to a computer via USB, allowing Python to process the RFID identification and motion data and send commands to the Arduino through serial communication at 9600 baud. The servo and LEDs responded according to the commands received from the Python program.

METHODOLOGY & PROGRAMMING LOGIC

The second experiment combined RFID authentication and motion detection to control a servo-based access system. The following steps describe the experimental procedure.

1. The RFID reader, MPU6050 sensor, servo motor and indicator LEDs were connected to the Arduino according to the specified pin assignments.

2. The Arduino program was uploaded to initialize the RFID reader, servo motor, LEDs, and serial communication.

3. The Python program was executed on the computer, establishing serial communication with the Arduino at 9600 baud.

```
void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();
```

4. An RFID card or tag was presented to the RFID reader. The reader obtained the card's unique identifier (UID) and transmitted it to the computer.

5. The Python program compared the received UID against a predefined list of authorized RFID identifiers.

```
AUTHORIZED_CARDS = [
    "0013082958",
```

6. If the UID was authorized, the program prompted the user to perform a circular hand gesture using the MPU6050 sensor. The motion data were analyed to determine whether the required gesture had been performed.

7. If both the RFID authentication and motion verification were successful, the Python program sent the character `A` to the Arduino. The servo moved to 180°, the green LED was switched on, and the red LED was switched off.

```
if (command == 'A') {
  servo.write(180);       // Unlock servo[cite: 4]
  digitalWrite(4, HIGH);  // Green LED ON[cite: 4]
  digitalWrite(3, LOW);   // Red LED OFF[cite: 4]
```

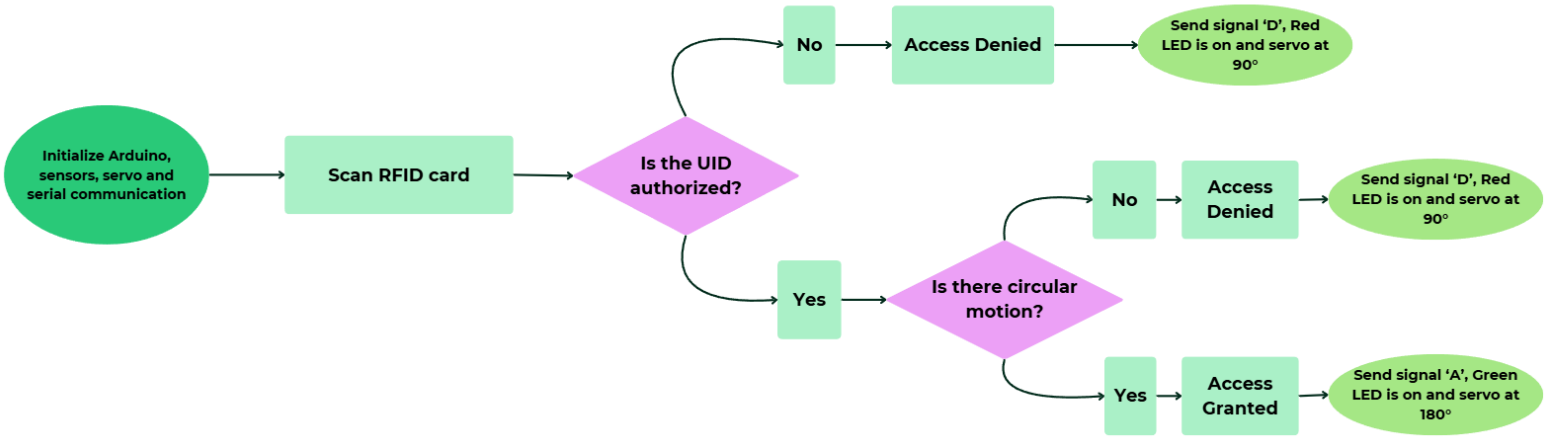
8. If the RFID UID was unauthorized or the motion was invalid, the program sent the character `D`. The servo returned to 90°, the red LED was switched on, and the green LED was switched off.

```
else if (command == 'D') {
  servo.write(90);         // Lock servo[cite: 4]
  digitalWrite(4, LOW);    // Green LED OFF[cite: 4]
  digitalWrite(3, HIGH);   // Red LED ON[cite: 4]
}
```

9. The test was repeated using different RFID cards and hand movements to observe the system's responses under authorized and unauthorized conditions.

10. The observed servo positions, LED states, RFID responses, and motion detection results were recorded for further analysis.

The overall communication sequence between the hardware and software can be summarized as follows.



Arduino code :

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo servo;

int servoPosition = 90;
unsigned long lastSendTime = 0;
const int sendInterval = 100; // Send sensor data every 100ms

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  servo.attach(9);       // Servo signal on D9[cite: 4]
  servo.write(servoPosition);

  pinMode(4, OUTPUT);    // Green LED on D4[cite: 4]
  pinMode(3, OUTPUT);    // Red LED on D3[cite: 4]
}

void loop() {
  // 1. Stream MPU6050 data periodically (every 100ms)
  if (millis() - lastSendTime >= sendInterval) {
    lastSendTime = millis();

    int ax, ay, az, gx, gy, gz;
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    // Print in the exact comma-separated format Python expects
    Serial.print(ax);
    Serial.print(",");
    Serial.print(ay);
    Serial.print(",");
    Serial.println(az);
  }
```

```
  // 2. Check if Python sent an access command ('A' or 'D')
  if (Serial.available() > 0) {
    char command = Serial.read();

    if (command == 'A') {
      servo.write(180);       // Unlock servo[cite: 4]
      digitalWrite(4, HIGH);  // Green LED ON[cite: 4]
      digitalWrite(3, LOW);   // Red LED OFF[cite: 4]
    }
    else if (command == 'D') {
      servo.write(90);        // Lock servo[cite: 4]
      digitalWrite(4, LOW);   // Green LED OFF[cite: 4]
      digitalWrite(3, HIGH);  // Red LED ON[cite: 4]
    }
  }
}
```

Phycharm code :

```
from collections import deque
import time
import serial
import matplotlib.pyplot as plt

# --- CONFIGURATION ---
SERIAL_PORT = "COM9"  # Update to match your Arduino's COM port
BAUD_RATE = 9600
MAX_POINTS = 100  # Rolling window for real-time motion path plotting
AUTHORIZED_CARDS = [
    "0013082958",

]  # Replace with your actual card UIDs


def detect_circular_motion(ser):
  """Captures live MPU6050 data and checks for circular motion patterns."""
  print("\n>>> Perform circular motion now (Monitoring sensor for 4 seconds)...")

  x_vals = deque(maxlen=MAX_POINTS)
  y_vals = deque(maxlen=MAX_POINTS)

  # Setup live plot window for motion verification
  plt.ion()
  fig, ax = plt.subplots(figsize=(5, 5))
  (line,) = ax.plot([], [], "ro-", alpha=0.7, label="Motion Path")
  ax.set_xlim(-18000, 18000)
  ax.set_ylim(-18000, 18000)
  ax.set_title("Verifying Circular Gesture...")
  ax.grid(True)

  start_time = time.time()
  motion_detected = False
```

```
  # Collect data for 4 seconds to analyze gesture
  while time.time() - start_time < 4.0:
    if ser.in_waiting > 0:
      line_data = ser.readline().decode("utf-8", errors="ignore").strip()
      if line_data:
        values = line_data.split(",")
        if len(values) >= 2:
          try:
            ax_val = int(values[0])
            ay_val = int(values[1])

            x_vals.append(ax_val)
            y_vals.append(ay_val)

            line.set_xdata(x_vals)
            line.set_ydata(y_vals)
            plt.draw()
            plt.pause(0.001)
          except ValueError:
            continue

  plt.close(fig)
  plt.ioff()

  # Simple gesture validation logic (e.g., checking if motion has spread across multiple quadrants)
  if len(x_vals) > 20:
    # A basic heuristic: if X and Y values fluctuate significantly, consider it a valid gesture
    x_range = max(x_vals) - min(x_vals)
    y_range = max(y_vals) - min(y_vals)
    if x_range > 3000 and y_range > 3000:
      motion_detected = True

  return motion_detected
```

```python
def main():
  try:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)  # Allow serial connection to initialize
    print("System Ready. Waiting for RFID scan...")

    while True:
      print("\n--------------------------------------------------")
      # USB RFID desktop reader behaves like a keyboard and types out the UID
      uid = input("Tap your RFID card on the USB Reader: ").strip().upper()

      if not uid:
        continue

      print(f"Scanned UID: {uid}")

      if uid in AUTHORIZED_CARDS:
        print("RFID Authorized! Please perform the motion gesture.")

        # Trigger motion verification using MPU6050
        if detect_circular_motion(ser):
          print("Motion verified successfully! Access Granted.")
          ser.write(b"A")  # Sends 'A' command to Arduino (Servo unlock + Green LED)
        else:
          print("Motion invalid or not detected. Access Denied.")
          ser.write(b"D")  # Sends 'D' command to Arduino (Red LED)
      else:
        print("Unauthorized RFID tag. Access Denied.")
        ser.write(b"D")  # Sends 'D' command to Arduino (Red LED)

      time.sleep(0.5)

  except KeyboardInterrupt:
    print("\nProgram terminated by user.")
  finally:
    if "ser" in locals() and ser.is_open:
      ser.close()


if __name__ == "__main__":
  main()
```

DATA COLLECTION & ANALYSIS

For task 2, the MPU6050 is combined with and RFID card reader as inputs for an access control system. The system moves a servo motor and switches on or off a red and green led based on the given inputs.

The condition-response pairs for the system are as follows:

RFID UID	Detected MPU6050 Motion	Char sent to Arduino	Servo Motor Angle (0°−180°)	Red LED State	Green LED State
Non-Registered UIDs	Circular or Non-circular	'D'	90	ON	OFF
0013082958	Circular	'A'	180	OFF	ON
0013082958	Non-Circular	'D'	90	ON	OFF

RESULTS & DISCUSSION

Hardware Behaviour and Evaluation Against Hypothesis

Real-Time Graphical Visualization: The matplotlib interactive plot managed to generate a continuous real time graph of the path of motion of the MPU6050. However, the precision of the plotted dots on the graph were below expectations causing a complete circular path of motion to be unable to be captured. During testing we found that even while the sensor was motionless, the dots would be scattered instead of being concentrated together due to its low precision. Consequently, the strictness of the circular motion detection for task 2 was relaxed to ensure the access control system functioned properly.

RFID Card Detection: The RFID card reader connected to the computer was able to read and input the card UID into the program. This UID was then able to be compared to a list of approved UIDs to decide whether to be accepted or rejected.

Actuator and Feedback response: Upon successful authentication through the RFID card reader and MPU6050 motion detector, the system delivers a response through the use of a servo motor and a red and greed LED. Through successful authentication, the python program sends the char 'A' to the Arduino program which through and if-else statement triggers the servo to move to the 180-degree position and set the red LED to LOW and the green LED to HIGH. Conversely, an unsuccessful authentication will prompt the python program to send the char 'D' to the Arduino program which tells the servo motor to remain at its starting position of 90-degree and to set the red LED to HIGH and the green LED to LOW.

Error sources and Solutions

Mitigating Low-Precision Hardware: The primary engineering challenge encountered during Part 2 integration stemmed from the uncalibrated sensor's poor baseline precision. Because raw accelerometer points scattered widely even during static positioning, strict gesture boundaries caused the access system to frequently misclassify valid user gestures as unauthorized attempts.

Compensatory Workaround: Rather than upgrading the hardware or implementing complex Kalman filtering outside the lab's scope, lowering the stringency of the spatial thresholds served as a practical, functional workaround. This adjustment balanced security parameters against physical hardware noise, ensuring reliable system usability while highlighting the operational trade-offs of low-cost inertial measurement units.

CONCLUSION

From this experiment, we have learnt how to interface an MPU6050 sensor and RFID module with an Arduino, while utilizing Python for real-time serial communication and motion pattern detection. The integration was successful, with the overall system fulfilling the established hardware and software objectives. The MPU6050 and USB RFID reader successfully communicated with the host system, allowing the continuous transmission of dynamic 2D accelerometer data and UID strings. Serial communication established at 9600 baud enabled a Python script to process the incoming sensor data and render real-time motion trajectories using matplotlib. The integrated logic successfully evaluated circular motion patterns alongside RFID credentials to authorize access; upon verification, the Python script sent an 'A' command to unlock a servo motor to 180° and illuminate a green LED, whereas unauthorized RFID tags or failed gesture executions correctly triggered a 'D' command to keep the servo locked at 90° and activate a red LED. These operational outcomes validate the feasibility of combining real-time gesture recognition with RFID technology to create a responsive mechatronic security system. To improve future system reliability, implementing software-side signal processing, such as a moving-average data smoothing filter, is required to mitigate inherent accelerometer drift and enhance motion tracking precision.

RECOMMENDATIONS

Based on the technical constraints identified during testing, several primary recommendations can improve the system's accuracy and reliability. To mitigate the MPU6050's inherent zero-g offset drift and reduce high-frequency background noise, software-side signal processing such as a digital low-pass or rolling moving-average filter should be implemented to smooth out trajectory tracking. Hardware stability can be improved by transitioning from standard breadboard wiring and jumper cables to soldered connections, which will minimize contact resistance and reduce the system's susceptibility to electromagnetic interference (EMI). Furthermore, refining the gesture recognition thresholds will allow the algorithm to handle baseline scatter more effectively, enabling the enforcement of strict circular motion criteria rather than relaxing the rules for authentication. Finally, the system's overall robustness would benefit from incorporating external power decoupling to mitigate servo switching transients and introducing strict frame delimitations within the serial data stream to prevent buffer synchronization drops between the Arduino microcontroller and the Python host environment.

REFERENCES

- Prahbu, S. (2026). What is RFID? How It Works? Interface RC522 RFID Module with Arduino https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/
- Tutorial, R. N. (2026). Security Access using MFRC522 RFID Reader with Arduino https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/
- Tutorial, R. N. (2026). Arduino Time AΣendance System with RFID https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/
- Sbx20. (2016). Arduino + MFRC522 RFID READER https://www.instructables.com/Arduino-MFRC522-RFID-READER/

APPENDICES

Video link:

https://drive.google.com/drive/folders/1rnCK8UPOOlMSEPXvnAnjwk3A3lNjo5Pu?usp=sharing

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: Harith Naufal Bin Ahmad Shukri Matric No: 2413381	Read [/] Understand [/] Agree [/]
	Name: Iman Bin Salman Matric No: 2418123	Read [/] Understand [/] Agree [/]
	Name: Luqman Hakimi Bin Suhaizi Matric No: 2411349	Read [/] Understand [/] Agree [/]
	Name: Muhamad Haziq Bin Mohd Shahril Matric No: 2418929	Read [/] Understand [/] Agree [/]